

Compte rendu : Mini Moteur MapReduce : MPI vs Apache Spark

Contexte et objectif

Ce projet (module Programmation Parallèle) compare deux frameworks de calcul distribué sur un problème classique de **WordCount** : **MPI** (via mpi4py, orchestration manuelle bas niveau) et **Apache Spark** (via PySpark, framework Big Data haut niveau). L'objectif est double : mesurer les performances brutes de chaque approche, et dégager des critères de choix pratiques (scalabilité, tolérance aux pannes, complexité de code, overhead de démarrage).

Contrairement aux notebooks précédents qui contenaient toute la logique de calcul inline, celui-ci fonctionne comme un **orchestrateur** : chaque cellule appelle, via subprocess, des scripts externes (mpi/mapreduce.py, mpi/benchmark.py, spark/wordcount_spark.py, spark/benchmark_spark.py) qui ne sont pas eux-mêmes présents dans le notebook. Le notebook se contente de lancer ces scripts et d'afficher/agréger leurs résultats.

1. Exécution MapReduce en MPI

python

```
subprocess.run(['mpirun', '-n', '4', 'python', 'mpi/mapreduce.py', 'data/big_input.txt'], ...)
```

Lance le pipeline Map → Shuffle → Reduce sur 4 processus MPI, appliqué à data/big_input.txt.

La sortie de cette cellule est vide (aucun texte imprimé, pas d'erreur signalée) — soit le script mapreduce.py n'affiche rien par conception (il écrit peut-être directement un résultat dans un fichier), soit la cellule a été exécutée sans capturer de sortie utile. Ce point mériterait d'être vérifié : pour un TP, il est préférable de montrer explicitement le résultat du comptage de mots (même partiellement) pour prouver que le calcul distribué a bien fonctionné.

2. Benchmark MPI — 1, 2 et 4 processus

Processus Temps

1	0,172 s
2	0,123 s
4	0,170 s

Le passage de 1 à 2 processus apporte un gain réel (~28% plus rapide), mais passer à 4 processus **annule ce gain** et revient quasiment au temps séquentiel. C'est un résultat cohérent avec un phénomène classique en calcul parallèle : au-delà d'un certain nombre de processus, l'**overhead de communication** (ici le *shuffle* MPI, qui redistribue les données entre processus) dépasse le gain de calcul, surtout sur un jeu de données de taille modeste. Le notebook ne commente pas ce plafonnement, alors que c'est justement le résultat le plus intéressant de cette section.

3. WordCount Spark

Spark terminé en 15,495 secondes

Top 5 mots : le: 300, mpi: 200, avec: 200, souris: 100, parallel: 100

Le calcul aboutit et affiche un résultat exploitable (top 5 des mots), contrairement à la section MPI équivalente. Le temps de 15,5 s est cohérent avec le coût de démarrage d'une JVM Spark, largement documenté dans la littérature comme un point faible de Spark sur de petits jeux de données.

4. Benchmark Spark (Cellules 7-8)

Spark [small] : 5,99 secondes

Spark [big] : 3,207 secondes

Résultat contre-intuitif non commenté : le fichier "big" est traité plus vite (3,2 s) que le fichier "small" (6,0 s). C'est l'inverse de ce qu'on attendrait. Deux explications possibles à vérifier :

- Une session Spark déjà initialisée (JVM déjà chaude) au moment du test "big", ce qui masquerait l'overhead de démarrage vu sur le test "small" — plausible si les deux benchmarks tournent dans le même processus Python.

- Une éventuelle inversion des labels "small"/"big" dans benchmark_spark.py. Dans les deux cas, ce point mérite une investigation avant de tirer des conclusions sur la scalabilité de Spark.

5. Graphiques comparatifs

Deux graphiques bien construits :

- **Scalabilité MPI** (courbe temps vs nombre de processus) avec une ligne horizontale de référence au niveau du temps Spark "big" — permet de visualiser à quel nombre de processus MPI atteindrait la performance de Spark.
- **Barres comparatives** MPI (1/2/4 proc) vs Spark (big), avec étiquettes de valeurs sur chaque barre — lisible et pédagogique.

Le code est propre (Matplotlib avec légendes, couleurs cohérentes, sauvegarde en PNG à 150 dpi) et suit les bonnes pratiques déjà vues dans le premier TP (labels clairs, grille en transparence, export automatisé).

6. Tableau comparatif final

C'est le point le plus problématique du notebook : le tableau de synthèse contient des valeurs **qui ne correspondent pas** aux mesures obtenues plus haut dans le même notebook :

Critère	Valeurs du tableau final	Valeurs réellement mesurées plus haut
Temps petit fichier	MPI ~0,8 s / Spark ~11 s	MPI : 0,123–0,172 s ; Spark WordCount : 15,5 s
Temps grand fichier	MPI ~1,1 s / Spark ~5 s	Spark big : 3,207 s (pas de "grand fichier" MPI mesuré séparément)

Ces chiffres semblent être des **valeurs génériques ou d'un exemple type**, pas les résultats effectivement produits par l'exécution du notebook. C'est le genre d'incohérence qu'un correcteur repère immédiatement : le tableau doit être régénéré à partir des CSV réellement sauvegardés (results/bench_mpi.csv, results/bench_spark.csv) plutôt que d'utiliser des chiffres écrits en dur.

7. Conclusion du notebook

La conclusion qualitative est solide et correspond à des critères de choix reconnus dans la littérature (MPI pour le HPC/contrôle fin, Spark pour le Big Data/simplicité/tolérance aux pannes). Mais elle répète le tableau avec les mêmes valeurs approximatives non vérifiées ("Meilleur"/"Lent"/"Acceptable" sans chiffres précis cette fois), ce qui est plus défendable en soi (jugement qualitatif) mais aurait gagné à être appuyé sur les vraies mesures du notebook.

Synthèse — Points forts

- Architecture d'orchestration propre (scripts externes appelés via subprocess, séparation calcul/analyse).
- Visualisations de bonne qualité, cohérentes avec les autres notebooks du dossier.
- Grille de critères de comparaison pertinente et bien choisie (performance, lignes de code, scalabilité, tolérance aux pannes, overhead).
- Bon réflexe de benchmark : test MPI à plusieurs niveaux de parallélisme (1/2/4 processus).